

Politechnika Gdańska

Katedra Optoelektroniki i Systemów Elektronicznych

Organizacja i Oprogramowanie Systemów Elektronicznych

Michał Kowalewski

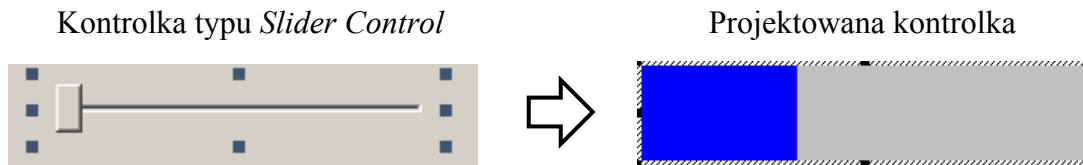
Projektowanie kontrolerek ActiveX



Politechnika Gdańska
2011

1. Wprowadzenie

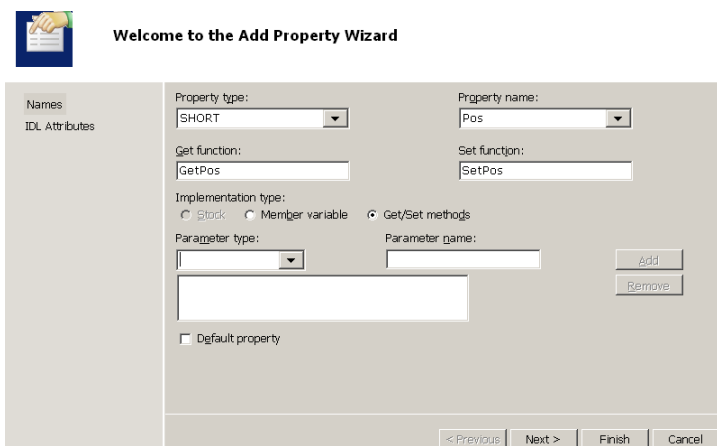
Celem ćwiczenia jest zaznajomienie z metodologią projektowania i wykorzystania w aplikacji kontrolki ActiveX. Projektowana kontrolka będzie pełniła funkcjonalność zbliżoną do systemowej kontrolki typu *Slider Control*.

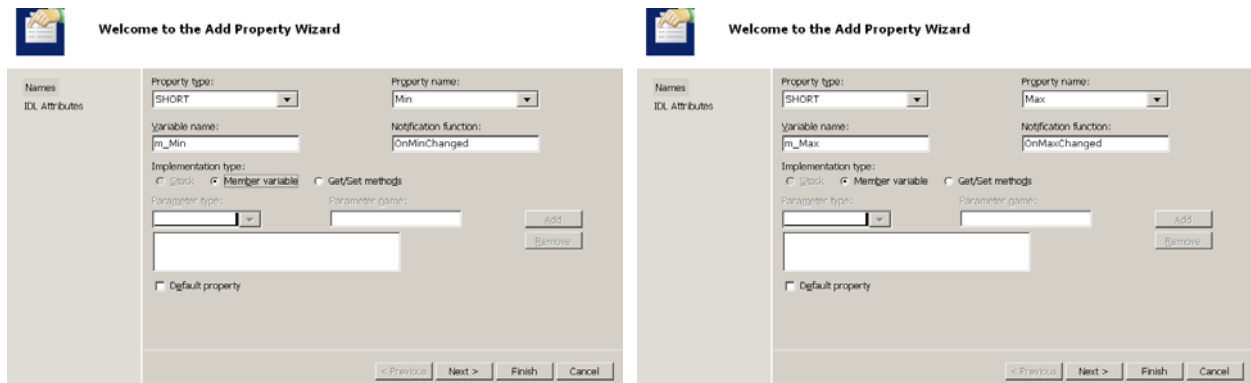


Programowanie realizowane jest w środowisku Microsoft Visual Studio.

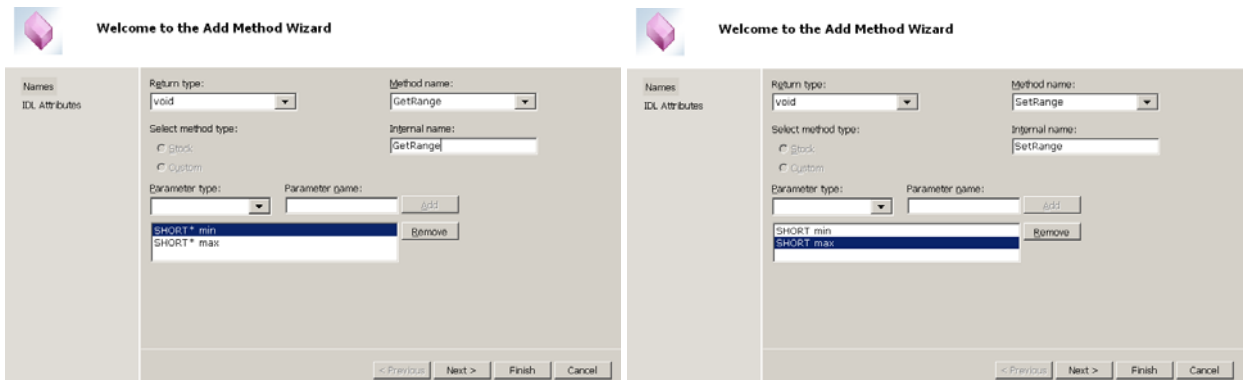
2. Zadania do wykonania

1. Uruchomić środowisko MS Visual Studio 2005 lub 2008.
2. W menu **View** wybrać następujące okna: *Solution Explorer*, *Class View*, *Resource View*, *Output*, *Toolbox*, *Properties Window*. Pozostałe okna zamknąć.
3. Utworzyć nowy projekt kontrolki ActiveX bazujący na bibliotece MFC:
 - a. Uruchomić kreatora aplikacji:
File/New/Project → Project Types/Visual C++/MFC/MFC ActiveX Control
 - b. W polu **Name** wpisać nazwę programu i kliknąć OK.
W nalszej części instrukcji nazwa programu została onaczona jako *AX1*.
 - c. Wybrać **Finish** aby utworzyć kontrolkę.
4. Korzystając z zakładki *Solution Explorer* oraz okna edycji kodu przeanalizować strukturę utworzonego projektu. Jakie jest przeznaczenie klas *CAX1Ctrl* i *CAX1PropPage*?
5. Skompilować projekt (**klawisz F7**).
6. Uruchomić program *ActiveX Control Test Container* (menu **Tools**).
7. Wybrać Edit/Insert New Control... i otworzyć wygenerowaną kontrolkę. Znaleźć fragment kodu programu odpowiedzialny za rysowanie widocznej na ekranie figury geometrycznej.
8. Dodawanie właściwości, metod i zdarzeń do kontrolki:
 - a. Zdefiniować w klasie **CAX1Ctrl** 3 właściwości typu **SHORT**: **Pos**, **Min** i **Max**.
W tym celu przejść do zakładki **Class View**, wybrać **AX1Lib/_DAX1** i z menu kontekstowego (dla każdej właściwości) wybrać opcję **Add/Add Property...**

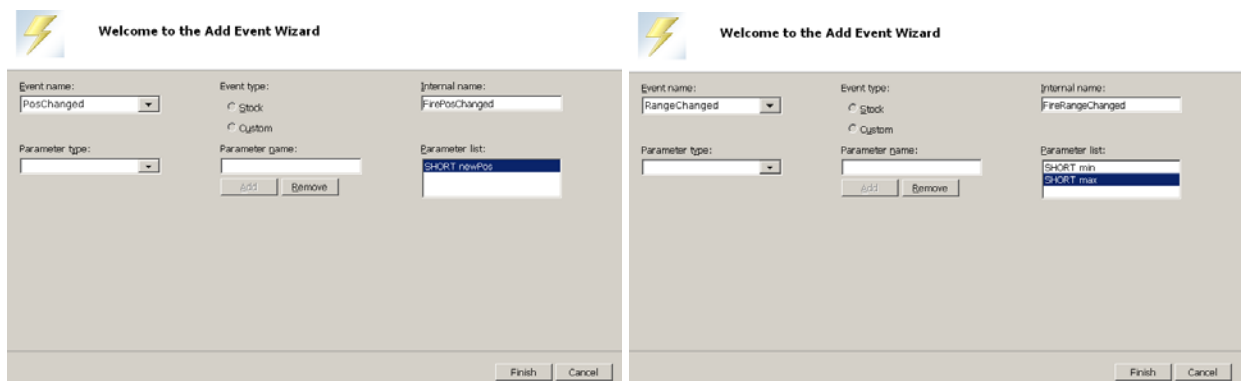




- b. Dodać do klasy **CAX1Ctrl** metody **GetRange** i **SetRange**, których zadaniem będzie odczyt i ustawianie wartości właściwości **Min** i **Max**.
W tym celu przejść do zakładki **Class View**, wybrać **AX1Lib/_DAX1** i z menu kontekstowego wybrać opcję **Add/Add Method...**



- c. Zdefiniować zdarzenia **PosChanged** i **RangeChanged** generowane przez kontrolkę przy aktualizacji zmiennych **Pos**, **Min** i **Max**. W tym celu przejść do zakładki **Class View**, wybrać klasę **CAX1Ctrl** i z menu kontekstowego wybrać opcję **Add/Add Event...**



9. Zdefiniowanie funkcji kontrolujących wartości właściwości **Pos**, **Min** i **Max**.

Zakładamy, iż bieżące położenie suwaka **Pos** spełnia warunek $\text{Min} \leq \text{Pos} \leq \text{Max}$.

Próba przypisania zmiennej **Pos** wartości spoza tego zakresu powinna skutkować skorygowaniem wartości do wartości granicznej **Min** lub **Max**. W tym celu należy zdefiniować w klasie **CAX1Ctrl** funkcję **CheckRange** o następującej postaci

```

bool CAX1Ctrl::CheckPos(void) {
    BOOL modified = FALSE;
    if (m_Pos < m_Min) {
        m_Pos = m_Min; modified = TRUE;
    } else if (m_Pos > m_Max) {
        m_Pos = m_Max; modified = TRUE;
    }
    return modified;
}

```

Kolejną kwestią jest sprawdzenie warunku **Min < Max**. Jeżeli nie jest on spełniony wówczas przyjmujemy wartości domyślne: **Min = 0, Max = 100**. W tym celu definiujemy w klasie **CAX1Ctrl** funkcję **CheckRange** o następującej postaci

```

bool CAX1Ctrl::CheckRange(void) {
    BOOL modified = FALSE;
    if (m_Min >= m_Max) {
        m_Min = 0; m_Max = 100; modified = TRUE;
    }
    return modified;
}

```

10. Dodać do klasy **CAX1Ctrl** zmienne typu **SHORT**: **m_Pos**, **m_Pos0** i **m_Width**.

11. Edycja kodu źródłowego

- W funkcji **OnResetState()** dodać inicjalizację właściwości domyślnymi wartościami: **Min = 0, Pos = 50, Max = 100**.
- W funkcji **GetPos()** zamienić „**return 0;**” na „**return m_Pos;**”
- Uzupełnić funkcję **SetPos()** dodając po **//TODO** następujące instrukcje

```

m_Pos = newVal;
if (CheckPos()) FirePosChanged(m_Pos);
InvalidateControl();

```

- Uzupełnić funkcję **OnMinChanged()** i **OnMaxChanged()** dodając po **//TODO** te same instrukcje

```

if (CheckRange()) FireRangeChanged(m_Min, m_Max);
if (CheckPos()) FirePosChanged(m_Pos);
InvalidateControl();

```

- Uzupełnić funkcję **GetRange()** dodając po **//TODO** następujące instrukcje

```

*min = m_Min;
*max = m_Max;

```

- Uzupełnić funkcję **SetRange()** dodając po **//TODO** następujące instrukcje

```

m_Min = min;
m_Max = max;
if (CheckRange()) FireRangeChanged(m_Min, m_Max);
if (CheckPos()) FirePosChanged(m_Pos);
InvalidateControl();
SetModifiedFlag();

```

- g. Zmodyfikować funkcję OnDraw() w sekcji TODO.

```
// szerokość kontrolki
m_Width = rcBounds.right - rcBounds.left;
// obliczenie położenia suwaka względem okienka kontrolki
int centerPos = m_Width * (m_Pos-m_Min) / (m_Max-m_Min);
// suwak stanowi dwa przylegające do siebie prostokąty
CRect selBounds = rcBounds, notSelBounds = rcBounds;
selBounds.right = centerPos;
notSelBounds.left = centerPos;

CBrush lSelBrush;
lSelBrush.CreateSolidBrush( RGB(0, 64, 224) );
pdc->SelectObject(&lSelBrush);
pdc->FillRect(selBounds, &lSelBrush);

CBrush lNotSelBrush;
lNotSelBrush.CreateSolidBrush( RGB(224, 224, 224) );
pdc->SelectObject(&lNotSelBrush);
pdc->FillRect(notSelBounds, &lNotSelBrush);
```

12. Dodać do projektu funkcje obsługi zdarzeń

WM_MOUSEMOVE i WM_LBUTTONDOWN i WM_LBUTTONUP.

W tym celu przejść do zakładki **Class View**, wybrać klasę **CAX1Ctrl**, z menu kontekstowego wybrać **Properties** i przejść do zakładki **Messages**. Dodać funkcje **OnMouseMove**, **OnLButtonDown** oraz **OnLButtonUp**.

- a. Uzupełnić kod źródłowy funkcji **OnMouseMove** dodając po TODO następujące instrukcje

```
if ((nFlags & MK_LBUTTON) == MK_LBUTTON) {
    m_Pos = m_Min + (m_Max - m_Min) *
        (point.x + m_Width / (m_Max - m_Min) / 2) / m_Width;
    CheckPos();
    if (m_Pos != m_Pos0) {
        m_Pos0 = m_Pos;
        SetModifiedFlag();
        InvalidateControl();
    }
}
```

- b. Uzupełnić kod źródłowy funkcji **OnLButtonDown** dodając po TODO następujące instrukcje

```
SetCapture();
m_Pos = m_Min + (m_Max - m_Min) *
    (point.x + m_Width / (m_Max - m_Min) / 2) / m_Width;
CheckPos();
if (m_Pos != m_Pos0) {
    m_Pos0 = m_Pos;
    SetModifiedFlag();
    InvalidateControl();
}
```

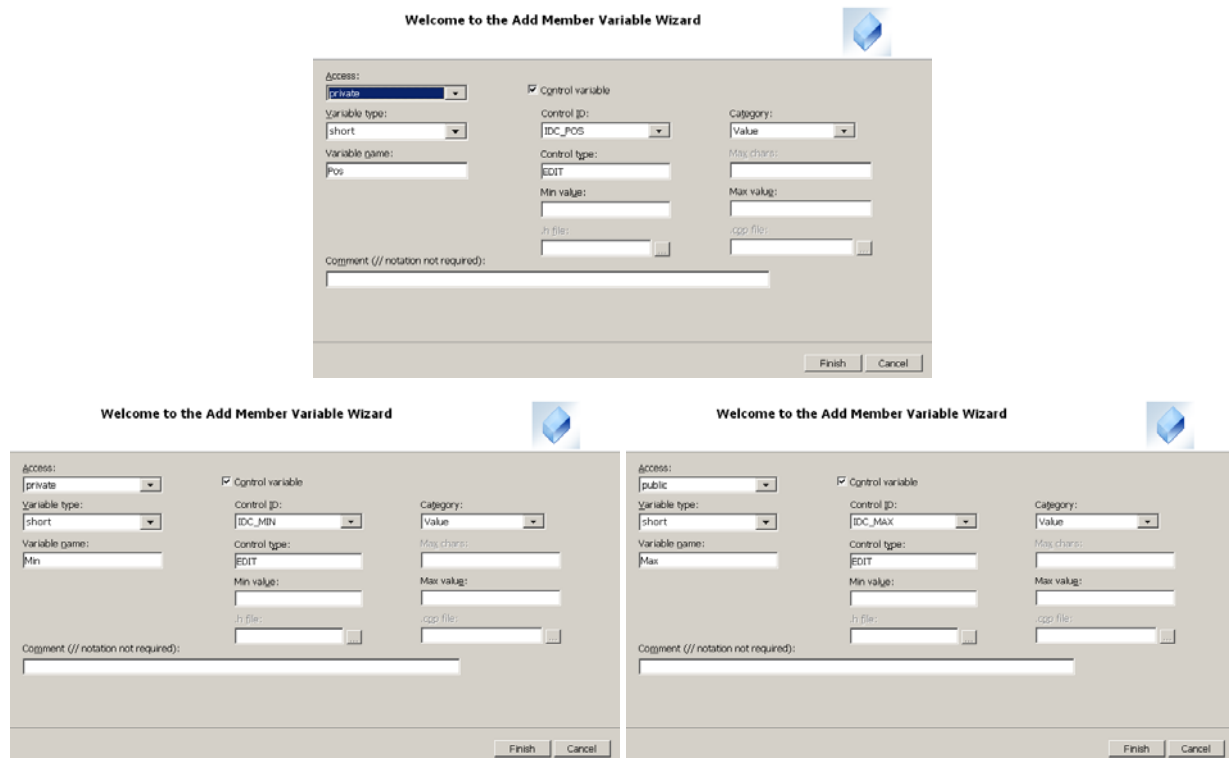
- c. Uzupełnić kod źródłowy funkcji **OnLButtonUp** dodając po TODO instrukcję

```
ReleaseCapture();
```

13. Skompilować projekt i przetestować działanie kontrolki za pomocą programu *ActiveX Control Test Container*. Sprawdzić działanie metod **GetRange**, **SetRange**, **GetPos**, **SetPos**.

14. Edycja okna właściwości

- Przejsć do zakładki **Resource View** i otworzyć okno dialogowe o identyfikatorze **IDD_PROPPAGE_AX1**.
- Umieścić na panelu 3 kontrolki **Static Text** i określić ich nazwy w polu **Caption** jako **Min**, **Pos** i **Max**.
- Umieścić na panelu 3 kontrolki **Edit Control** i zdefiniować ich identyfikatory jako: **IDC_MIN**, **IDC_POS** i **IDC_MAX**.
- Dodać do kontrolek zmienne typu **SHORT**: **Pos**, **Min** i **Max**



15. Edycja kodu źródłowego

- Dodać do funkcji **DoDataExchange** klasy **CAX1PropPage** następujące instrukcje:

```
DDP_Text(pDX, IDC_POS, pos, _T("Pos") );
DDP_Text(pDX, IDC_MIN, Min, _T("Min") );
DDP_Text(pDX, IDC_MAX, Max, _T("Max") );
```

*Funkcja **DDP_PostProcessing(pDX)** powinna być wywołana na samym końcu!*

- Dodać do funkcji **DoPropExchange** klasy **CAX1Ctrl** następujące instrukcje:

```
PX_Short(pPX, _T("Min"), (short &m_Min, 0);
PX_Short(pPX, _T("Pos"), (short &m_Pos, 50);
PX_Short(pPX, _T("Max"), (short &m_Max, 100);
```

16. Sprawdzić w dokumentacji MSDN przeznaczenie funkcji **DDX_**, **DDP_** i **PX_**.

17. Przetestować działanie okna właściwości w programie *ActiveX Control Test Container*.

Zadanie dodatkowe 1 - Adding "Stock" properties

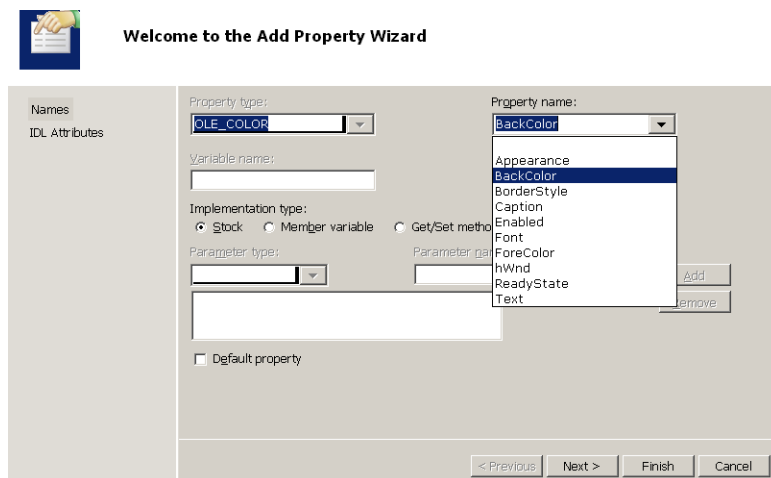
The term *stock properties* means that its one of a set of common properties that MFC code stores and initializes for you. The MFC code also performs appropriate action when the value of a stock property is changed. The ClassWizard provides code for changing nine (9) such properties. Since these properties are built-in, they are obviously the easiest to deal with in terms of the work required by the developer. As you'll see shortly, you literally don't add a single line of code to add stock property to your control!

The Visual Studio provides code for the following stock properties:

- o Appearance
- o BackColor
- o ForeColor
- o BorderStyle
- o Font
- o Caption
- o Enable
- o Text
- o hWnd

From these, we will work with the **BackColor** and **ForeColor** properties. We'll start by adding the *BackColor* stock property

1. Select AX1Lib/_DAX1 in **Class View** window. Next choose *Add/Add Property...*



2. Click *Finish* button to finalize the generation of the stock property. At this point, the ClassWizard will store the value of the BackColor property and initialize it to background color of the container window. The ClassWizard will also add the following line to your control's IDL file:

```
[id(DISPID_BACKCOLOR), bindable, requestedit] OLE_COLOR BackColor;
```

Finally, ClassWizard will also add the code to invalidate the control whenever the value of the BackColor property changes, thereby forcing the OnDraw function to redraw the control. The only thing you have to do is to use the color contained in the property to paint the background color of control.

- At this point, replace the following line of code in **OnDraw** function

```
lNotSelBrush.CreateSolidBrush( RGB(224, 224, 224) );
```



```
lNotSelBrush.CreateSolidBrush( TranslateColor( GetBackColor() ) );
```

- Now, let's add a Color property page to the program. This page will allow users of the control to change the BackColor our newly added property property at design-time. To do this, simply open the AX1Ctrl.cpp file and locate the // Property pages comment.

Once you've done that, you should see the following:

```
BEGIN_PROPPAGEIDS(CPlotCtrl, 1)
    PROPPAGEID(CPlotPropPage::guid)
END_PROPPAGEIDS(CPlotCtrl)
```

The first line tells the compiler how many pages exist. Notice that it's set to 1. Change this value to 2 as we're going to add a new page.

Now insert the following line just before the END_PROPPAGEIDS line

(The **CLSID_CColorPropPage** is defined automatically since this is a property page CLSID for a stock property).

```
PROPPAGEID(CLSID_CColorPropPage)
```

Once you've finished, your new property page id map should look like the following:

```
BEGIN_PROPPAGEIDS(CPlotCtrl, 2)
    PROPPAGEID(CPlotPropPage::guid)
    PROPPAGEID(CLSID_CColorPropPage)
END_PROPPAGEIDS(CPlotCtrl)
```

Once you make the above changes, the stock property page is automatically linked to the BackColor property.

- Now that you've seen how to add the BackColor stock property to an ActiveX control, follow these same steps in order to add the *ForeColor* property. **Note that you do not need to add a property page for the second property.**

- Replace the following line of code in **OnDraw** function

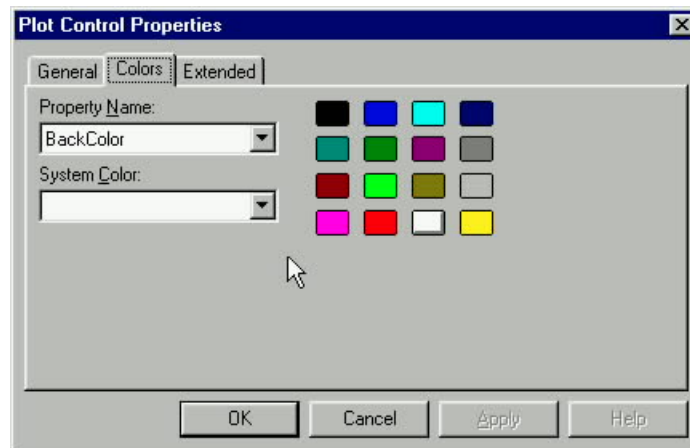
```
lSelBrush.CreateSolidBrush( RGB(0, 64, 224) );
```



```
lSelBrush.CreateSolidBrush( TranslateColor( GetForeColor() ) );
```


- After adding these stock properties, build your control and test it using ActiveX Test Container.

As you can see in the figure below, the ClassWizard has added the appropriate controls for changing the stock properties.



Zadanie dodatkowe 2 – Napisać program wykorzystujący utworzoną kontrolkę ActiveX.

1. Utworzyć nowy projekt bazujący na bibliotece MFC i oparty o okna dialogowe:
 - a. Uruchomić kreatora aplikacji:
File/New/Project → Project Types/Visual C++/MFC/MFC Application
 - b. W polu **Name** wpisać nazwę programu i kliknąć OK.
 - c. W kolejnych krokach wybrać rodzaj aplikacji: **Dialog Based**.
2. Wstawić na panelu dwie kontrolki: *Slider Control* i *AX1*.
3. Dodać zmienne typu kontrolnego do obu kontrollek.
4. Dodać do programu procedury obsługi zdarzenia zmiany położenia suwaka kontrollek *Slider Control* oraz *AX1*. W tym celu wybrać jedną z kontrollek, a następnie z menu kontekstowego wybrać opcję **Add Event Handler**.
W przypadku kontrolki *Slider Control* wybrać komunikat powiadomienia `NM_CUSTOMDRAW`, natomiast w przypadku kontrolki *AX1* zdarzenie `PosChanged`.
5. Uzupełnić kod źródłowy utworzonych funkcji. Kontrolka *SliderControl* powinna zmieniać położenie suwaka kontrolki *AX1* i odwrotnie zmiana położenia suwaka kontrolki *AX1* powinna aktualizować położenie suwaka kontrolki *SliderControl*.

